Учебный программный проект

Система автоматизации функций секретаря

Научно-технический отчёт

Реализация на C++17 и Qt Widgets

Объект исследования: имитационная модель календаря фирмы с ограниченными ресурсами, напоминаниями и стратегиями разрешения конфликтов.
Основание: исходное задание «Система автоматизации функций секретаря», документация проекта и уточнённые пользовательские требования.
Экспериментальная база: 120 реально выполненных прогонов; результаты сохранены в JSON и CSV. Данные реальных сотрудников и отделов не используются.

Версия приложения: `live-minute-v5`

5 ноября 2026 г.

Оглавление

1 Назначение и постановка задачи 2
1.1 Исходные требования и принятые уточнения 2
1.2 Функциональный результат 2

2 Архитектура приложения 3
2.1 Разделение ответственности 3
2.2 Взаимодействие классов 3
2.3 Данные и идентификаторы 4

3 Математическая модель и основные алгоритмы 4
3.1 Обозначения и связь с реализацией 4
3.2 Пересечение интервалов и ресурсный конфликт 5
3.3 Жизненный цикл и порядок обработки времени 6
3.4 Случайное появление событий 7
3.5 Длительность и заблаговременное планирование 9
3.6 Периодические семинары 10
3.7 Фильтр часов и назначения руководителей 11
3.8 Поиск вариантов разрешения конфликтов 12
3.9 Напоминания, повторения и журнал почты 12
3.10 Сохранение и восстановление состояния 14

4 Показатели и методика исследования 14
4.1 Определения показателей 14
4.2 План эксперимента 15

5 Экспериментальные результаты 17

6 Проверка корректности 19
6.1 Сборка и автоматические тесты 19
6.2 Граничные случаи 19

7 Ограничения и направления развития 20

8 Заключение 20

A Файлы результатов 21

1 Назначение и постановка задачи

Разработано локальное приложение для демонстрации автоматизации функций секретаря: ведения календаря, контроля занятости, формирования напоминаний, фиксации решений по конфликтующим заявкам и анализа результатов моделирования. Программа объединяет ручные действия пользователя и случайный сценарий работы фирмы.

Цель доработки заключалась в завершении уже существовавшей реализации без замены работающего календаря и анимации. Приоритетами были соответствие доступной спецификации, корректность, простота архитектуры и возможность объяснить алгоритмы средствами базового C++ и Qt. Сохранены роли пользователей, архив, публичные ID, 19 типов модельных событий, ручной и автоматический режимы.

1.1 Исходные требования и принятые уточнения

Исходное задание предоставлено пользователем в формате DOCX. Оно предусматривает 5–9 отделов, общий календарь, участников и помещения, изменение и отмену мероприятий, просмотр событий, уведомления по важности и разрешение ресурсных конфликтов. Период моделирования составляет 7–30 дней, шаг — 30 или 60 минут. Реализованные алгоритмы конкретизированы в README и MODEL.

Уточнения пользователя изменяют регулярное планирование: планёрки отделов создаются по очереди и размещаются на разных датах. Поэтому текущий пресет не означает ежедневную планёрку каждого отдела. Начальный календарь пуст, планы появляются во время прогона. Семинары возникают случайно и по отдельному периодическому шаблону. В интерфейсе доступны фильтры дат, часов внутри каждого дня, отдела и сотрудника. У каждого отдела хранится назначенный руководитель; секретарь может изменить назначение. Полномочия оператора ограничены выбранной ролью и разрешёнными отделами.

Электронная почта реализована как **имитация**. Реальных сетевых отправлений нет; журнал содержит адресатов, тему, текст, плановое время, отправку и доставку. Возможность использовать такую реализацию вместо настоящей электронной почты **необходимо согласовать с преподавателем**. Приложение является локальной учебной системой, а не промышленным почтовым сервисом.

1.2 Функциональный результат

Реализованы три стратегии конфликтов, повторные напоминания с исключением дубликатов, структурированная статистика, сохранение и воспроизведение состояния, пакетные эксперименты, экспорт

CSV/JSON. Будущие мероприятия доступны и за пределами горизонта моделирования. Настройки формы передаются в движок; относящиеся только к поиску переноса элементы отключаются при выборе стратегии «Отклонение заявки».

Удалены неиспользуемая форма `widget.ui`, устаревший статический загрузчик планёрок и маркеры слияния Git в файле исключений. Пользовательское удаление столбцов напряжения, навыка, нагрузки и репутации сохранено. Сами внутренние эффекты этих показателей не удалены: они нужны для динамики учебной модели.

2 Архитектура приложения

2.1 Разделение ответственности

Проект разделён на библиотеки `calendar_core` и `calendar_ui`. Ядро зависит только от Qt Core. Интерфейс использует Qt Widgets и не выполняет вместо сервисов проверку конфликтов, вычисление напоминаний или подсчёт статистики. Точки входа: графическая `Modeling` и консольная `modeling_experiments`.

CalendarService отвечает за данные календаря, права и историю. SimulationEngine управляет модельными минутами, командами и состояниями. ConflictResolver выполняет поиск свободного интервала. ReminderService планирует уведомления, а SimulatedEmailChannel регистрирует имитируемую доставку. SimulationReport строит статистику и экспорт, SimulationStore вместе с JsonCodec сохраняет и проверяет состояние. ExperimentRunner последовательно выполняет независимые прогоны. SimulationWindow и RunTools представляют данные пользователю; OfficeAnimator отвечает только за движение элементов сцены.

Композиция применяется для объектов одного прогона. Наследование введено там, где есть реальный контракт поведения: `SimulatedEmailChannel` реализует `NotificationChannel::send`. Замена способа доставки не требует изменения алгоритма напоминаний. Для трёх стратегий конфликтов отдельные однотипные классы не нужны: выбор политики и общий поиск вариантов проще и короче.

2.2 Взаимодействие классов

SimulationWindow владеет экземпляром SimulationEngine. Движок содержит CalendarService, ReminderService и SimulatedEmailChannel, обращается к ConflictResolver при возникновении конфликта. ReminderService использует полиморфный контракт NotificationChannel, реализованный SimulatedEmailChannel.

SimulationStore восстанавливает движок через публичные команды, а SimulationReport только читает его состояние. ExperimentRunner в каждый момент владеет одним текущим движком; завершённые прогоны сворачиваются в показатели для сравнения. Такое разделение исключает прямое изменение календаря из виджетов и сохраняет простую последовательность обработки.

2.3 Данные и идентификаторы

`EventData` содержит редактируемые сведения. `CalendarEvent` дополняет их ID, статусом, версиями и историческими кодами участников. Внутренний ID сотрудника неизменен; публичный код вида `1_01` содержит отдел и порядковый номер. При переводе меняется публичный код активных записей, но не архивные снимки. Это позволяет сохранить корректные ссылки и объяснимую историю.

3 Математическая модель и основные алгоритмы

3.1 Обозначения и связь с реализацией

Мероприятие i задаётся началом s_i, концом e_i, множеством участников U_i и необязательным помещением r_i. Момент создания обозначается c_i, текущее модельное время — t, начало прогона — t_0, горизонт — H. В формулах разности времён измеряются в секундах, если единица не указана отдельно. Продолжительности мероприятий в алгоритме генерации измеряются целым числом минут.

Формулы описывают текущую реализацию, а не дополнительную оптимизационную модель. Для каждого алгоритма указаны методы, выполняющие соответствующие действия.

Как читать математические обозначения. Индекс у буквы уточняет объект: s_i — начало мероприятия с номером i, t_n — время после шага с номером n. Запись $f(t)$ означает значение величины или функции при времени t; это не обязательно отдельная функция в программе. Например, $\rho(t)$ — рассчитанная доля занятости к выбранному моменту. Основные операции читаются следующим образом:

- $\min(a, b)$ выбирает меньшее из двух чисел, $\max(a, b)$ — большее. Например, $\min(8, 5) = 5$, $\max(8, 5) = 8$. В алгоритмах эти операции ограничивают время горизонтом и не позволяют длительности стать меньше допустимого минимума.
- $\lfloor x \rfloor$ — округление вниз до целого: $\lfloor 3.8 \rfloor = 3$. $\mathrm{round}(x)$ — округление до ближайшего целого: $\mathrm{round}(3.8) = 4$. Для положительных полуце-

лых значений используемая в программе функция qRound округляет вверх: $\text{round}(3.5) = 4$.

- $a \bmod b$ — остаток от целочисленного деления a на b. Например, $7 \bmod 6 = 1$, а $6 \bmod 6 = 0$. Эта операция позволяет циклически возвращаться к первому отделу после последнего.
- $\sum$ — суммирование. Запись $\sum_{r=1}^{n} x_r$ означает $x_1 + x_2 + \ldots + x_n$. Под знаком суммы указывается, какие элементы складываются.
- $\sqrt{x}$ — квадратный корень, а x^2 — квадрат числа. Они используются при расчёте разброса результатов эксперимента.
- $\wedge$ означает логическое «и»: должны выполняться оба условия. $\vee$ означает «или»: достаточно хотя бы одного, допускается выполнение обоих.
- $\cap$ означает пересечение множеств: общих участников или общую часть временных интервалов. $\emptyset$ — пустое множество; $\neq$ — «не равно». Поэтому $U_i \cap U_j \neq \emptyset$ означает, что у двух мероприятий есть общий участник.
- $\in$ читается «принадлежит». Например, $\Delta \in \{30, 60\}$ означает, что шаг выбирается из двух значений — 30 или 60 минут. Квадратная скобка интервала включает границу, круглая исключает: $[s_i, e_i)$ включает начало, но не включает конец. Вертикальные черты $|I|$ в формулах занятости обозначают длительность интервала в секундах.

Вероятностные обозначения раскрыты непосредственно рядом с формулами генерации, чтобы их смысл можно было связать с поведением приложения.

3.2 Пересечение интервалов и ресурсный конфликт

Занятость представлена полуоткрытым интервалом $I_i = [s_i, e_i)$. Условие пересечения двух интервалов имеет вид

$$O_{ij} = (s_i < e_j) \wedge (s_j < e_i). \tag{1}$$

Конфликт существует при пересечении времени и наличии общего ресурса:

$$C_{ij} = O_{ij} \wedge \big((U_i \cap U_j \neq \emptyset) \vee (r_i \neq \emptyset \wedge r_i = r_j)\big). \tag{2}$$

Здесь $\emptyset$ для помещения означает отсутствие назначенной комнаты. Встречи 10:00–11:00 и 11:00–12:00 совместимы: в момент 11:00 ресурс освобождается.

Алгоритм проверки и записи. В `CalendarService::checkEvent` и `validate` выполняются следующие действия:

1. Проверить пользователя и допустимость операции над существующей записью.
2. Проверить непустое название, существование участников и помещения, корректность интервала, отсутствие повторяющихся участников и доступность ресурсов.
3. Для каждого активного мероприятия проверить C_{ij}, пропуская изменяемую запись.
4. При ошибке вернуть причину и идентификаторы конфликтующих мероприятий. При успехе создать или изменить запись, повысить нужные версии и сохранить снимок истории.

Проверка предшествует записи: отклонённое изменение не уничтожает прежний вариант. Секретарь имеет право изменения календаря; руководитель ограничен разрешёнными отделами; обычный сотрудник не получает право изменения через этот сервис.

3.3 Жизненный цикл и порядок обработки времени

Для неотменённого мероприятия при достижении соответствующих моментов выполняются переходы

$$\text{Planned} \xrightarrow{t=s_i} \text{InProgress} \xrightarrow{t=e_i} \text{Completed}. \tag{3}$$

Отмена допустима из Planned и приводит к Cancelled. Завершённые и отменённые записи переносятся в архив; история сохраняет точное время каждого изменения. Событие, которое заканчивается за горизонтом, не завершается искусственно. Начало в точности на H также не исполняется в текущем прогоне.

Английские слова в формуле — имена состояний в коде: Planned означает «ожидает исполнения», InProgress — «выполняется», Completed — «завершено», Cancelled — «отменено». Стрелка обозначает смену состояния, надпись над ней — условие по модельному времени.

Следующий внешний шаг определяется формулой

$$t_{n+1} = \min(t_n + 60\Delta, H), \qquad \Delta \in \{30, 60\}. \tag{4}$$

Внутри шага `SimulationEngine::step` вызывает `minute` для каждой минуты. При первом шаге дополнительно обрабатывается исходный момент t_0.

В этой формуле Δ (дельта) — длина шага в минутах; множитель 60 переводит её в секунды. Выбор минимума не позволяет следующему шагу выйти за H. Например, если до горизонта осталось 10 минут, последний шаг длится 10 минут, даже когда в настройках выбрано 30.

Алгоритм модельной минуты.

1. Продвинуть календарь к t, обработать завершения и опубликовать новые записи истории. Начала в точности на t временно отложить.
2. Если $t < H$ и минута рабочая, проверить срок создания регулярного плана, сгенерировать случайные поступления и изменения расписания.
3. Отправить наступившие напоминания по актуальным версиям мероприятий.
4. Обработать начала в точности на t и опубликовать новые переходы.

В `CalendarService::advanceTo` переходы сортируются по времени; при равном времени завершение выполняется перед началом, затем используется порядок ID. На границе горизонта не выполняются генерация и доставка уведомлений.

Реальная скорость интерфейса регулируется независимо:

$$\tau(v) = \max\{1, \mathrm{round}(\tau_0 / v)\}, \qquad v \in \{0.5, 1, 2, 5\}, \tag{5}$$

где τ_0 и τ — интервалы QTimer в миллисекундах.

Буква τ читается «тау», а v — выбранный множитель скорости. При $\tau_0 = 250$ мс и $v = 2$ таймер вызывает шаг каждые 125 мс. Операция max не допускает интервал меньше 1 мс. Это пример расчёта настройки, а не измерение длительности выполнения шага. Формула реализована в `SimulationWindow::updateSpeed`. Пауза останавливает таймер, но не изменяет модельные часы. Анимация представляет результат шага.

3.4 Случайное появление событий

Пусть λ — среднее число случайных поступлений за рабочий день фирмы, а M — число рабочих минут в дне. В каждую рабочую минуту генерируется

$$N_t \sim \mathrm{Poisson}(\mu), \qquad \mu = \lambda / M, \qquad \Pr(N_t = k) = e^{-\mu} \frac{\mu^k}{k!}, \quad k = 0, 1, \ldots \tag{6}$$

Что означают Poisson и Pr. Poisson читается «распределение Пуассона». Это правило случайного выбора количества поступлений за фиксированный промежуток: 0, 1, 2 и так далее. Оно используется, чтобы события не появлялись через одинаковые промежутки времени. Задание среднего значения 6 за день не означает ровно шесть поступлений каждый день.

N_t — случайное число поступлений в минуту t. Символ $\sim$ читается «имеет распределение», поэтому $N_t \sim \mathrm{Poisson}(\mu)$ означает: количество

выбирается по распределению Пуассона с параметром μ (мю). Этот параметр равен среднему числу поступлений за минуту. Буква λ (лямбда) обозначает среднее число за рабочий день.

Pr — общепринятое обозначение вероятности, от английского probability. $\Pr(N_t = k)$ читается: «вероятность того, что за эту минуту поступит ровно k событий». Вероятность лежит между 0 и 1; для перевода в проценты её умножают на 100. Здесь $k!$ — факториал: произведение целых чисел от 1 до k; например, $3! = 1 \cdot 2 \cdot 3 = 6$, по определению $0! = 1$. Буква e без индекса обозначает математическую константу, приблизительно равную 2.71828; она отличается от e_i, обозначающего конец мероприятия. $e^{-\mu}$ — эта константа в степени $-\mu$.

Пример расчёта вероятности. При рабочем дне 09:00–18:00 получаем $M = 9 \cdot 60 = 540$ минут. Если $\lambda = 6$, то $\mu = 6/540 \approx 0.0111$. Вероятность хотя бы одного поступления за конкретную рабочую минуту равна $1 - e^{-6/540} \approx 0.01105$, то есть примерно 1.105%. Большинство минут проходят без поступлений; это согласуется со средним уровнем шесть за день. Пример иллюстрирует формулу и не является новым экспериментальным результатом. Величина N_t учитывает и происшествия, поэтому число принятых календарных мероприятий может быть меньше числа поступлений. Следовательно, $\Pr(N_t \geq 1) = 1 - e^{-\lambda/M}$; допускаются как пустые минуты, так и несколько поступлений за одну минуту. Изменения расписания получают отдельную выборку с параметром $\lambda_{\text{change}}/M$. В нерабочие минуты эти поступления не генерируются. Используются `std::mt19937` и стандартные распределения C++.

`std::mt19937` — генератор псевдослучайных чисел: он вычисляет последовательность из начального значения seed. Seed можно понимать как номер начальной последовательности. При одинаковом seed, совместимой реализации генератора и одинаковом порядке действий результат повторяется. Само распределение Пуассона используется через `std::poisson_distribution`; Pr в отчёте — математическое обозначение, а не отдельная вызываемая функция C++.

Тип выбирается по относительным весам:

$$\Pr(K_t = k) = \frac{\widetilde{w}_k(t)}{\sum_j \widetilde{w}_j(t)}. \tag{7}$$

Здесь K_t — выбранный тип поступления, k — один из типов, $\widetilde{w}_k(t)$ — его вес с учётом текущего состояния. Волнистая черта над w подчёркивает, что используется скорректированный вес. Знаменатель складывает веса всех типов. Вес задаёт относительную частоту, а не готовый процент. Например, для трёх допустимых типов с весами 2, 3 и 5 сумма равна 10, поэтому вероятности выбора составляют 20, 30 и 50 процентов. Нулевой вес исключает тип из выбора. В программе такой выбор

выполняет `std::discrete_distribution`. Базовые веса заданы в `eventCatalog`. Для мгновенного происшествия без допустимых объектов вес становится нулевым. Для начала конфликта вес дополнительно умножается на $1+q/K$, где q — число перегруженных отделов, K — общее число отделов. Вес обычного планируемого мероприятия не обнуляется при нехватке участников: его попытка создания может закончиться зарегистрированным отказом. Важность выбирается аналогично по трём весам из настроек.

Условия допустимости происшествий. Метод `candidates` формирует список подходящих объектов, после чего `trigger` выбирает один из них равновероятно и применяет действие. Выздоровление возможно не ранее суток после заболевания. Завершение конфликта и восстановление помещения — не ранее часа после соответствующего происшествия. Новый конфликт невозможен в отделе с уже существующим конфликтом. Поломка не выбирается для помещения с текущим мероприятием. Болезнь и перевод выбирают доступного сотрудника, не занятого текущим мероприятием.

3.5 Длительность и заблаговременное планирование

Базовая длительность d_0 задана типом мероприятия. Если в отделе организатора есть конфликт, применяется увеличение; затем учитывается навык организатора a:

$$d_1 = d_0 + z\left\lfloor \frac{d_0}{2} \right\rfloor, \qquad d = \max\left\{15, d_1 - \left\lfloor \frac{d_1 a}{500} \right\rfloor\right\}, \tag{8}$$

где $z = 1$ при активном конфликте и $z = 0$ иначе. Округление вниз соответствует целочисленному делению в `SimulationEngine::schedule`. Навык остаётся внутренним параметром модели; показателя в пользовательской форме нет. Если d превышает весь рабочий день, попытка отклоняется.

Для любого нового плана и редактирования существующего действует ограничение

$$s_i - c_i \geq 5 \cdot 3600. \tag{9}$$

При редактировании c_i — время первоначального создания, а не время последней правки. Дополнительно нельзя переносить начало в прошлое относительно текущего t.

При случайном создании выбирается число рабочих дней предварительного сдвига D:

$$D = \begin{cases} 0, & \text{с вероятностью } 0.3, \\ \mathrm{UnifInt}(L_{\min}, L_{\max}), & \text{с вероятностью } 0.7. \end{cases} \tag{10}$$

UnifInt означает равномерный выбор целого числа включительно.

В отличие от распределения Пуассона, здесь число выбирается из ограниченного диапазона с одинаковыми шансами для всех его значений. Например, UnifInt$(1, 5)$ возвращает 1, 2, 3, 4 или 5; вероятность каждого значения равна $1/5 = 20\%$. Это пояснение относится к самому равномерному выбору внутри дальней ветви, которая применяется с вероятностью 70%. Фигурная скобка в формуле означает выбор одной из двух ветвей, а не их сложение. В коде равномерный выбор реализован через `std::uniform_int_distribution`. По умолчанию дальний диапазон равен 1–5 рабочим дням. Нулевой сдвиг не отменяет пятичасовой минимум. Выходные и недостаток свободной части дня могут увеличить календарный интервал.

Для рассматриваемой рабочей даты q пусть b_q и f_q — начало и конец рабочего дня. Тогда границы допустимого начала равны

$$A_q = \max(b_q, c_i + 5 \cdot 3600), \qquad B_q = f_q - 60d. \tag{11}$$

Если $A_q > B_q$, рассматривается следующая рабочая дата. На первой подходящей дате

$$J \sim \text{UnifInt}\left(0, \left\lfloor \frac{B_q - A_q}{60} \right\rfloor\right), \qquad s_i = A_q + 60J, \qquad e_i = s_i + 60d. \tag{12}$$

Далее выбираются помещение и важность, а сформированная заявка передаётся в `submit`. До s_i принятое мероприятие остаётся в статусе Planned.

Регулярные мероприятия. Метод `scheduleMeetingCreation` выбирает случайную рабочую минуту очередной даты создания. В `dailyMeetings` отделы обходятся циклически:

$$k_{\text{next}} = (k \bmod K) + 1. \tag{13}$$

Здесь k — текущий номер отдела, K — количество отделов, индекс next означает «следующий». Например, при $K = 6$ после отдела 5 выбирается отдел 6, а после отдела 6 — отдел 1. Планёрка создаётся на ближайшую последующую рабочую дату, не занятую другой регулярной планёркой и удовлетворяющую пятичасовому минимуму. Время исполнения планёрки — начало рабочего дня, длительность — 30 минут. Недельное совещание руководителей создаётся отдельно. Предварительного заполнения всего календаря нет.

3.6 Периодические семинары

Шаблон задаёт период P в календарных днях, заблаговременность L, время начала h и длительность d. По умолчанию $P = 7$, $L = 2$, $h = 14{:}00$,

$d = 90$ минут. Начальный календарь пуст. Первая плановая дата равна дате начала моделирования плюс L. В рабочую минуту t очередная заявка создаётся, когда

$$\mathrm{date}(t) + L \geq q_n. \tag{14}$$

Здесь q_n — очередная плановая дата, а date извлекает дату из модельного момента. Если дата нерабочая или до начала меньше пяти часов, она сдвигается вперёд до допустимой даты q'_n:

$$q'_n = \min\{q \geq q_n : \ q \text{ рабочая}, \ (q, h) \geq t + 5 \text{ часов}\}, \qquad q_{n+1} = q'_n + P. \tag{15}$$

Обозначение (q, h) означает момент с датой q и временем h. Поиск минимума реализован простым переходом к следующему дню. Заявка получает конференц-зал, среднюю важность и сотрудников минимум двух отделов. Сначала выбирается по одному доступному сотруднику разных отделов, затем состав дополняется до заданного минимума, не превышая максимум. При нехватке участников регистрируется отказ.

Каждый срок обрабатывается один раз, в том числе при отказе. Поиск переноса использует общую стратегию конфликтов и не меняет дату следующей заявки шаблона. Созданный семинар остаётся ожидающим до начала; работают обычные напоминания, архивирование и эффект обучения при завершении. Будущие даты за горизонтом доступны в календаре. Случайные семинары продолжают генерироваться независимо от шаблона.

3.7 Фильтр часов и назначения руководителей

Часовой фильтр задаётся полуоткрытым интервалом $[a, b)$, где $a < b$, и применяется к каждой дате выбранного периода. Сначала интервал мероприятия обрезается общим диапазоном дат. Оно попадает в выборку, если существует выбранная дата q, для которой

$$\max(s_i, (q, a)) < \min(e_i, (q, b)). \tag{16}$$

Здесь s_i, e_i — границы уже обрезанного интервала мероприятия. Для проверки достаточно даты его начала либо следующей даты, если время начала уже достигло b. Сканировать все дни длинного мероприятия не требуется. При отображении сетки пересечение проверяется для каждой её даты, поэтому мероприятие через полночь не появляется в неподходящем дневном окне. Фильтр применим также к архиву.

В отделе хранится внутренний идентификатор руководителя. Изначально назначен первый сотрудник. Секретарь выбирает нового руководителя из сотрудников этого отдела. Назначение влияет на последующее формирование недельных и случайных совещаний руководителей; состав ранее созданных мероприятий сохраняется. Болезнь временно

исключает руководителя из новых планов, но не меняет назначение. Перевод в другой отдел освобождает прежнюю должность; произвольная автоматическая замена не выполняется. Команда назначения входит в журнал воспроизведения, а итоговые назначения — в отчёт.

3.8 Поиск вариантов разрешения конфликтов

В `ConflictResolver::alternatives` для каждой рассматриваемой рабочей даты перебираются начала

$$s' = b_q + 1800m, \quad m = 0, 1, \ldots, \qquad s' \geq \max(s_i, t), \quad s' + (e_i - s_i) \leq f_q. \tag{17}$$

Горизонт включает исходную дату и заданное число последующих рабочих дней. Сначала проверяется исходное помещение, затем остальные в порядке ID. Для мероприятия без помещения альтернативные комнаты не добавляются.

Алгоритм обработки заявки.

1. Сохранить исходные параметры заявки и проверить права, календарь, пятичасовой минимум и минутную точность времени.
2. При отсутствии ошибки выполнить создание или изменение.
3. При ресурсном конфликте и стратегии «Отклонение заявки» зарегистрировать отказ.
4. Для стратегий «Предложение вариантов» и «Автоматический перенос» перебрать даты, начала и помещения, проверяя каждый вариант тем же календарным сервисом. Остановить поиск после получения заданного максимума предложений.
5. При стратегии «Предложение вариантов» сохранить найденные варианты и ожидать выбора без бронирования ресурсов. Выбранный вариант повторно подать на проверку. При стратегии «Автоматический перенос» сразу использовать первый допустимый вариант. Если вариантов нет, сохранить отказ.

Длительность и участники сохраняются, чужие мероприятия не перемещаются. Для регулярной планёрки исключаются даты, уже занятые другой планёркой пресета. Сетка 30 минут означает поиск среди этих кандидатов, а не доказательство отсутствия любого свободного интервала между узлами сетки.

3.9 Напоминания, повторения и журнал почты

Пусть L_i — срок первого уведомления в календарных днях, P_i — период в часах, а u_i — время создания записи или изменения версии расписания.

Тогда

$$F_i = s_i - 86400L_i, \qquad p_i = 3600P_i, \qquad N_i = \max(F_i, u_i). \tag{18}$$

Номинальные сроки образуют последовательность $F_i + kp_i < s_i$, $k \geq 0$. Начальная установка N_i обеспечивает одно компенсирующее уведомление при позднем создании, не отправляя все пропущенные сроки.

Уведомление отправляется при $N_i \leq t$, $N_i < s_i$, статусе Planned и $t < H$. После отправки курсор обновляется:

$$N_i \leftarrow F_i + \left(\left\lfloor \frac{N_i - F_i}{p_i} \right\rfloor + 1 \right) p_i. \tag{19}$$

Стрелка $\leftarrow$ означает присваивание нового значения переменной. Разность $N_i - F_i$ показывает, сколько времени прошло от первого номинального срока; деление на p_i и округление вниз находят номер уже достигнутого периода. Прибавление единицы переводит курсор к следующему сроку. Например, при $F_i =$ 09:00, периоде два часа и компенсирующей отправке $N_i =$ 10:30 следующий срок будет 11:00, а не 12:30: сохраняется исходная сетка повторов. Пример предполагает, что само мероприятие начинается позже 11:00. Следующее значение строго больше прежнего. Курсор хранится для пары ID/версия расписания, поэтому повторная проверка времени не отправляет тот же срок снова.

Алгоритм уведомления.

1. `ReminderService::observe` получает новые записи истории. Для новой версии пересчитывает курсор; при неизменной версии сохраняет его.
2. `deliver` проверяет наступление срока и статус мероприятия. Для каждого уникального участника создаёт запись отправки.
3. Почтовый канал фиксирует сообщение и равенство $t_{\text{delivered}} = t_{\text{sent}}$, затем служба передвигает курсор.
4. Отмена, начало или завершение мероприятия удаляют его будущие уведомления.

Изменения начала, конца, важности или множества участников увеличивают `scheduleVersion`. Простое переименование не начинает новую серию. При ручном изменении на паузе обработка выполняется в следующем модельном тике; плановый срок и фактический модельный момент отправки сохраняются раздельно.

Тема и текст письма фиксируются на момент отправки и не меняются при последующих правках календаря. Адреса имеют вид `employee<ID>@work`. В `RunTools::refresh` доставки группируются по ключу

$$g = (\text{eventId}, \text{scheduleVersion}, \text{dueTime}, \text{sentAt}). \tag{20}$$

Здесь g — набор из четырёх значений, по которому объединяются записи: `eventId` — номер мероприятия, `scheduleVersion` — версия его расписания, `dueTime` — плановый срок уведомления, `sentAt` — модельное время отправки. Скобки обозначают совместный ключ, а не арифметическое действие. Одна группа отображается одной строкой со списком адресатов; повторные рассылки образуют другие группы. Полный текст открывается в отдельном окне. Статистика и экспорт продолжают учитывать доставку каждому участнику.

3.10 Сохранение и восстановление состояния

Файл содержит настройки, seed, версию генератора, журнал внешних действий, контрольный календарь и SHA-256 от сериализованного отчёта и состояния генератора. Автоматические действия повторно вычисляются движком из исходных параметров.

Алгоритм загрузки в SimulationStore.

1. Проверить формат файла, версию генератора и структуру журнала действий.
2. Создать отдельный движок с сохранёнными настройками.
3. Последовательно применить записанные действия через публичные методы движка.
4. Сравнить полученные журнал, календарь и контрольную сумму с сохранёнными.
5. Передать восстановленный движок интерфейсу только после успешной проверки.

Повреждённый файл не подменяет текущий прогон. QSaveFile выполняет атомарную замену файла при записи. Формат состояния 2 учитывает текст писем; формат 1 поддерживается через проверку прежнего представления почты. Контрольная сумма не является электронной подписью. Совместимость стандартных распределений C++ между разными библиотеками не гарантируется; несовпадение результата приводит к отказу загрузки.

4 Показатели и методика исследования

4.1 Определения показателей

Счётчики мероприятий разделяют создание, начало, завершение, отмену и текущее ожидание. Заявки разделяют принятие, отказ, ожидание

и выбор варианта. Выполняется баланс

$$Q = Q_{\text{accepted}} + Q_{\text{rejected}} + Q_{\text{pending}} + Q_{\text{selected}}. \tag{21}$$

Здесь Q — число зарегистрированных попыток; остальные величины — числа принятых, отклонённых, ожидающих и ранее выбранных попыток соответственно. При стратегии «Предложение вариантов» выбор создаёт новую проверяемую попытку; поэтому общее число заявок не является одинаковым внешним потоком для всех стратегий. Конфликт людей и комнаты может увеличить два причинных счётчика, но общий счётчик конфликтных заявок — один раз.

Для помещения j рабочая занятость Bj — сумма пересечений фактически начавшихся мероприятий с прошедшими рабочими интервалами. Номинальный рабочий фонд Wj включает периоды ремонта. Использование фонда: Пусть $\mathcal{W}(t)$ — объединение рабочих интервалов от t_0 до t, $\mathcal{E}_j$ — множество начавшихся мероприятий помещения j, а $|I|$ — длина временного множества в секундах. Тогда

$$B_j(t) = \sum_{i \in \mathcal{E}_j} \left| [s_i, e_i) \cap [t_0, t) \cap \mathcal{W}(t) \right|, \qquad W_j(t) = |\mathcal{W}(t)|. \tag{22}$$

Доли занятости помещения и фирмы рассчитываются как

$$\rho_j(t) = \frac{B_j(t)}{W_j(t)}, \qquad \rho(t) = \frac{\sum_j B_j(t)}{\sum_j W_j(t)}. \tag{23}$$

Буква ρ читается «ро» и обозначает долю занятости. Индекс j указывает помещение. Выражение $\sum_j$ означает сложение по всем помещениям; $\sum_{i \in \mathcal{E}_j}$ в предыдущей формуле — сложение по начавшимся мероприятиям этого помещения. Пересечение с $\mathcal{W}(t)$ оставляет только прошедшие рабочие часы. Например, занятость 2 часа из 8 рабочих часов даёт долю $2/8 = 0.25$, или 25%. Это иллюстрация расчёта, а не значение из проведённой серии. В интерфейсе доля умножается на 100%. Реализация находится в `SimulationReport::metrics`; рабочие секунды выделяет функция `workingSeconds`. При нулевом фонде сохраняется 0 и явное значение доступных секунд 0. Будущие брони не увеличивают фактическую занятость. Дополнительно экспортируется полная длительность занятости, включая нерабочие часы, и срез текущего состояния.

4.2 План эксперимента

Приведённая экспериментальная серия получена на генераторе `live-minute-v4`, до добавления периодического шаблона семинаров и явных назначений руководителей. Она сохранена как результат предыдущей версии; её числовые выводы не выдаются за измерения версии v5. Новые механики проверяются отдельными автоматическими сценариями, описанными далее.

Выполнено шесть серий по 20 запусков: стратегии «Отклонение заявки», «Предложение вариантов» и «Автоматический перенос» при интенсивностях 6 и 12 событий в рабочий день. Каждая серия обозначена полным названием стратегии и интенсивностью. Дата начала — 12 октября 2026 г.; период — 14 дней; 6 отделов по 8 сотрудников; 7 помещений; рабочее время 09:00–18:00, понедельник–пятница; шаг — 30 минут; seed — 42–61.

Для стратегии «Предложение вариантов» в пакетном эксперименте задано правило: после каждого шага выбрать первый вариант ожидающей заявки. В обычном интерфейсе решение принимает человек. Увеличение интенсивности относится ко всем случайным поступлениям, включая происшествия, а не только к бронированиям. Начальный набор регулярных планов включён.

Среднее и выборочное стандартное отклонение рассчитаны по завершённым запускам:

$$\overline{x} = \frac{1}{n}\sum_{r=1}^{n} x_r, \qquad s_x = \sqrt{\frac{1}{n-1}\sum_{r=1}^{n}(x_r - \overline{x})^2}, \quad n > 1. \tag{24}$$

Черта над x означает среднее значение. s_x — выборочное стандартное отклонение, то есть характеристика разброса результатов отдельных прогонов вокруг среднего. Оно имеет те же единицы, что и исходный показатель. Для его расчёта из каждого результата вычитается среднее, разность возводится в квадрат, квадраты складываются, сумма делится на $n - 1$, затем извлекается квадратный корень. Делитель $n - 1$ соответствует используемой выборочной оценке дисперсии. Малое отклонение означает близкие результаты прогонов; большое — заметный разброс. Здесь x_r — значение показателя в прогоне r, n — число завершённых прогонов серии. При единственном завершённом прогоне программа выводит отклонение 0; это техническое значение, а не оценка разброса по одному наблюдению. В `ExperimentRunner::report` обрабатываются числовые показатели верхнего уровня; вложенный список отдельных помещений не усредняется этим циклом.

Алгоритм пакетного эксперимента. Метод `comparison` сочетает три политики с исходной и удвоенной интенсивностью, получая шесть сценариев. Для повторения r, считая с нуля, используется $\text{seed}_r = \text{seed}_0 + r$. Метод `advance` выполняет один шаг текущего движка. В пакетном режиме стратегии «Предложение вариантов» после шага до окончания прогона выбирается первый вариант каждой ожидающей заявки. При достижении горизонта сохраняются настройки, seed и показатели, затем создаётся движок следующего прогона. Агрегаты включают только завершённые прогоны; неполная серия помечается соответствующим при-

знаком. В тексте приведены средние и стандартные отклонения; они не являются доверительными интервалами. Скрипт подготовки результатов независимо пересчитывает средние, отклонения и балансы из строк запусков, проверяя совпадение с агрегатами программы.

Одинаковые seed не гарантируют одинаковый поток поступлений между стратегиями: изменение состояния влияет на допустимые события и порядок случайных решений. Следовательно, исследование сравнивает полную динамическую модель. Проверка статистической значимости и доказательство изолированного причинного эффекта не входят в выводы.

5 Экспериментальные результаты

Все численные данные ниже получены из сохранённого файла `experiments/results/comparison.json`; числовые описания автоматически сформированы скриптом `make_results.py`. Ручных демонстрационных чисел нет.

«Отклонение заявки», интенсивность 6. Выполнено 20 прогонов. Среднее число созданных мероприятий — 44.10, завершённых — 32.55 (выборочное стандартное отклонение 3.47), отказанных попыток — 8.55 (отклонение 2.54). Среднее число заявок — 61.50, конфликтных — 7.55. Доставлено в среднем 329.95 имитируемых писем. Занятость помещений — 3.95% (стандартное отклонение 0.55 процентного пункта).

«Предложение вариантов», интенсивность 6. Выполнено 20 прогонов. Среднее число созданных мероприятий — 50.70, завершённых — 37.05 (выборочное стандартное отклонение 4.83), отказанных попыток — 1.05 (отклонение 0.89). Среднее число заявок — 71.30, конфликтных — 9.80. Доставлено в среднем 392.50 имитируемых писем. Занятость помещений — 4.49% (стандартное отклонение 0.69 процентного пункта).

«Автоматический перенос», интенсивность 6. Выполнено 20 прогонов. Среднее число созданных мероприятий — 50.70, завершённых — 37.05 (выборочное стандартное отклонение 4.83), отказанных попыток — 1.05 (отклонение 0.89). Среднее число заявок — 61.50, конфликтных — 9.80. Доставлено в среднем 393.25 имитируемых писем. Занятость помещений — 4.49% (стандартное отклонение 0.69 процентного пункта).

«Отклонение заявки», интенсивность 12. Выполнено 20 прогонов. Среднее число созданных мероприятий — 68.70, завершённых — 42.05 (выборочное стандартное отклонение 7.12), отказанных попыток — 24.50 (отклонение 5.70). Среднее число заявок — 103.70, конфликтных — 22.10. Доставлено в среднем 427.75 имитируемых писем. Занятость помещений — 5.41% (стандартное отклонение 1.20 процентного пункта).

«Предложение вариантов», интенсивность 12. Выполнено 20 прогонов. Среднее число созданных мероприятий — 89.40, завершённых — 54.35 (выборочное стандартное отклонение 10.30), отказанных попыток — 3.05 (отклонение 3.91). Среднее число заявок — 134.45, конфликтных — 30.70. Доставлено в среднем 580.85 имитируемых писем. Занятость помещений — 7.20% (стандартное отклонение 1.55 процентного пункта).

«Автоматический перенос», интенсивность 12. Выполнено 20 прогонов. Среднее число созданных мероприятий — 89.60, завершённых — 54.65 (выборочное стандартное отклонение 10.48), отказанных попыток — 2.85 (отклонение 3.87). Среднее число заявок — 103.75, конфликтных — 30.00. Доставлено в среднем 583.00 имитируемых писем. Занятость помещений — 7.25% (стандартное отклонение 1.58 процентного пункта).

Сравнение при интенсивности 12. Среднее число завершённых мероприятий увеличилось с 42.05 при стратегии «Отклонение заявки» до 54.65 при стратегии «Автоматический перенос» (на 12.60, или 30.0%). Число отказанных попыток снизилось с 24.50 до 2.85. Эти различия относятся к выполненным запускам; статистическая значимость и причинный эффект не заявляются.

Поиск свободных вариантов снижает число окончательных отказов, но может увеличить число подтверждённых будущих мероприятий, отменяемых затем из-за болезни или поломки. Поэтому число созданных записей нельзя отождествлять с числом завершённых задач. Невысокая средняя занятость помещений объясняется масштабом демонстрационной интенсивности, отказами, отменами и наличием нескольких комнат; она не является оценкой производительности реальной фирмы.

Результаты стратегий «Предложение вариантов» и «Автоматический перенос» близки, поскольку в пакетном опыте используется одинаковое предпочтение первого варианта. Отличие моментов принятия решения и повторной проверки может менять отдельные исходы и ко-

личество уведомлений. При другом поведении оператора выводы для стратегии «Предложение вариантов» могли бы отличаться.

6 Проверка корректности

6.1 Сборка и автоматические тесты

Сборка выполнена через CMake. Проверены ядро, графическое приложение и консольные эксперименты. Тесты используют Qt Test; GUI запускается через offscreen, без физического дисплея. Проверки выполнены на Qt 6.4.2 и Qt 5.15.13. Подробные журналы проверки сохранены в `experiments/verification`.

Набор calendarservice проверяет права, интервалы и архив; simulationengine — генерацию, зависимости и сроки; completion — напоминания, сохранение текста писем, совместимость состояния и эксперименты; calendarwindow — формы, управление, анимацию, будущие мероприятия и просмотр групповых сообщений.

Проверка воспроизведения включает сохранение после ручных изменений и случайных событий, загрузку, сравнение полного отчёта, продолжение обеих моделей до горизонта и повторное сравнение. Отдельно проверяются повреждённая контрольная сумма и несовместимая версия генератора. Совпадение результатов при шагах 30/60 минут проверяется для одинакового сценария без различающихся внешних решений.

6.2 Граничные случаи

Проверяются точная граница пяти часов, соприкосновение интервалов, отсутствие свободного времени, недопустимые роли, отмена после появления уведомлений, изменение важности, сохранение текущего курсора при переименовании, начало за горизонтом, некорректные настройки и повторное принятие предложения. Дополнительно проверяются часовые границы и переход через полночь, повторение семинаров через семь дней, перенос с выходных, конфликт помещения без повторной генерации одной заявки, назначение руководителя, запрет назначения сотрудника другого отдела и освобождение должности при переводе. Сохранение и продолжение сверяются также для периодических семинаров и ручных назначений, в том числе при шагах 30 и 60 минут.

Исправленные в ходе проверки тестовые ожидания касались двух разных уведомлений на разных сроках (компенсирующего и очередного) и вложенного экранирования JSON в CSV. Алгоритм не был изменён только ради совпадения с неверным ожиданием теста.

7 Ограничения и направления развития

Программа является учебной имитационной моделью с демонстрационными весами и эффектами. Нет эмпирической калибровки на данных фирмы, реального SMTP, внешней авторизации, многопользовательского сервера или базы данных. Адреса фиктивны. Возможность имитации почты требует отдельного согласования с преподавателем.

Сохранение основано на повторном выполнении команд: это удобно для объяснения, но загрузка длительна для больших сценариев и зависит от совместимости генератора. Поддерживаются версии состояния 1, 2 и 3; произвольная миграция других форматов не предусмотрена. Контрольная сумма обнаруживает расхождение, но не подтверждает подлинность источника данных. Новые прогоны используют генератор v5; сохранения v4 воспроизводятся по прежним правилам совещаний, без периодических семинаров. Повторное сохранение обновляет формат, сохраняя алгоритм v4. Для новых механик следует создать новый прогон.

Время модели имеет минутную точность, фиксированное смещение UTC+03:00 и не моделирует время пути сотрудников. Ограничение одной сетки календаря — 366 дней; полный список мероприятий сохраняет доступ ко всем записям. В качестве дальнейшей работы можно рассмотреть калибровку вероятностей, фиксацию независимого потока случайных решений и реализацию дополнительного почтового канала после согласования требований.

8 Заключение

Существующий проект доведён до связанного цикла работы: настройка, генерация и ручное планирование, разрешение конфликтов, исполнение, уведомления, архив, статистика, сохранение и воспроизведение. Модель отделена от интерфейса и может выполняться в консольных экспериментах. Полезные механики и пользовательские изменения сохранены.

Экспериментальная серия содержит 120 завершённых запусков с исходными данными и проверяемыми агрегатами. На выбранных параметрах поиск допустимого переноса уменьшил число отказов и увеличил число завершённых мероприятий. Границы применимости выводов указаны явно; имитация почты не выдаётся за реальную доставку.

A Файлы результатов

- `comparison.json`: все строки запусков, конфигурации, средние и отклонения;
- `comparison.csv`: плоская таблица для внешнего анализа;
- `example-state.json`: воспроизводимое состояние одного демонстрационного прогона;
- `example-report.json`, `example-report.csv`: подробный отчёт того же прогона;
- `configuration.json`: исходная конфигурация;
- `data_manifest.json`: проверенные источники числовых описаний отчёта.

Источники

[1] Система автоматизации функций секретаря. Исходное задание, предоставленное пользователем в формате DOCX.

[2] Правила реализованной модели. Репозиторий проекта: `documentation/MODEL.md`.

[3] Архитектура и взаимодействие компонентов. `documentation/ARCHITECTURE.md`.

[4] Исходный код и автоматические тесты. Каталоги `src/`, `tests/`; версия генератора live-minute-v5 (экспериментальные данные: v4).

[5] Результаты экспериментальных запусков. `experiments/results/comparison.json`; 120 наблюдений.